



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

September 2026

CSA0613– Design and Analysis of Algorithms

**Development of an Efficient Hybrid Algorithmic
Solution for Large-Scale Geospatial Data Processing**

Assignment Report

Submitted by

A.Bhavya(192424417)

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.E./B.Tech. Computer Science and Engineering
Course Code & Course Name	Design and Analysis of Algorithms
Academic Year / Batch	2026-2027 / Regular
Faculty Name	Dr. F. Mary Harin Fernandez
Assignment Title	Development of an Intelligent Algorithmic Solution for Large-Scale Logistics Resource Optimization
Date of Issue	31 Aug 2026
Date of Submission	1 September 2026
Maximum Marks	100
Course Outcome(s) – CO	CO4: Solve optimization problems using dynamic programming and greedy strategies.
Bloom's Taxonomy Level	L4 – Analyze, L5 – Evaluate, L6 – Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Efficient logistics resource allocation with improved vehicle utilization, reduced computational cost, and scalable decision-making

B. Assignment Problem / Challenge

Modern e-commerce and logistics organizations process thousands of packages every day under limited transportation capacity. Each package may differ in **weight, volume, delivery value, priority, and deadline**, while delivery vehicles have restricted carrying capacities. Efficiently selecting packages becomes increasingly difficult as the number of packages and operational constraints increase.

Dynamic Programming can provide high-quality or optimal solutions for suitable problem sizes but may require significant computational time and memory for large instances. **Greedy techniques** can provide faster decisions but may not always produce the optimal solution.

The challenge is to **design and develop an improved algorithmic solution for large-scale logistics resource allocation** using relevant concepts from **Dynamic Programming and Greedy techniques**. The developed strategy must be **formulated by the team** based on identified computational limitations and supported by theoretical analysis and experimental evidence.

C. Problem Statement

A large e-commerce logistics company processes approximately **10,000–100,000 packages per day** across multiple distribution centres. Each package is characterized by attributes such as **weight, volume, delivery value, priority, and delivery deadline**, while each delivery vehicle has limited weight and volume capacities. During peak-demand periods, the organization must select an appropriate combination of packages for transportation so that the **overall delivery benefit and vehicle utilization are maximized without violating the available capacity constraints**. Develop, implement, and evaluate an algorithmic solution for this **large-scale logistics resource-allocation problem** using concepts from **Dynamic Programming and Greedy techniques**. Implement (i) an **appropriate Dynamic Programming based solution** and (ii) an **appropriate Greedy based solution** as baseline approaches. Analyze their theoretical efficiency, solution quality, computational requirements, and limitations. Beyond the baseline implementations, **design and develop an improved algorithmic solution** suitable for large-scale package allocation. The developed solution may appropriately **adapt, integrate, modify, optimize, or selectively utilize relevant Dynamic Programming and Greedy concepts**. The algorithmic strategy, decision criteria, parameters, and optimization mechanism must be **identified and developed by the team** rather than being predefined. Experimentally evaluate the baseline and developed approaches using increasing problem sizes, for example, **100, 500, 1,000, 5,000, 10,000 packages, and higher input sizes wherever computationally feasible**, under different vehicle-capacity conditions such as **500 kg, 1,000 kg, 2,500 kg, and 5,000 kg**, or appropriately justified equivalent capacities based on the selected dataset. Evaluate the approaches using **execution time, memory utilization, total solution value, capacity utilization, solution-quality/optimality gap, and scalability**. Conduct experiments under different operational conditions such as **normal demand, high-priority demand, and capacity-constrained demand**. Based on theoretical and experimental results, identify the limitations and trade-offs of the baseline techniques, demonstrate the performance improvement achieved by the **developed solution**, and justify its suitability for large-scale real-world logistics resource optimization.

Expected Development

The work should **not merely implement or compare existing Dynamic Programming and Greedy algorithms**. The solution should demonstrate algorithmic development through:

- Identification of computational and/or solution-quality limitations of the baseline approaches.
- Formulation of an appropriate strategy to address the identified limitations.
- **Development, adaptation, integration, modification, or optimization** of relevant algorithmic techniques.
- Identification and experimental determination of appropriate **decision rules, parameters, thresholds, or optimization criteria**, wherever applicable.
- Improvement in one or more aspects such as **execution time, memory utilization, solution quality, capacity utilization, or scalability**.
- Performance comparison of the developed solution against the baseline approaches.
- Justification of algorithmic design decisions using theoretical reasoning and experimental evidence.

D. Requirements and Constraints

The following requirements and constraints shall be considered:

- Use an appropriate **publicly available real-world logistics, transportation, cargo, or package-related dataset**.
- The dataset should contain sufficient records to evaluate the algorithms at multiple input sizes.
- Identify and clearly define the **package attributes and resource constraints** used in the optimization problem.
- Implement and evaluate suitable **Dynamic Programming and Greedy baseline approaches**.
- **Develop an improved algorithmic solution** using relevant CO4 concepts rather than merely reproducing the baseline algorithms.
- The developed strategy must be **designed and justified by the team**.
- Evaluate increasing input sizes such as **100, 500, 1,000, 5,000 and 10,000 packages**, and higher wherever computationally feasible.
- Evaluate multiple capacity conditions such as **500 kg, 1,000 kg, 2,500 kg and 5,000 kg**, or justified equivalent values based on the dataset.
- Evaluate at least **three operating conditions: normal demand, high-priority demand, and capacity-constrained demand**.
- Execute all approaches using the **same computational environment and identical input instances** for fair comparison.
- Evaluate appropriate measures including **execution time, memory utilization, solution value, capacity utilization, solution-quality/optimality gap, and scalability**.
- Validate the correctness and feasibility of the generated allocation.
- Clearly document the **dataset source, experimental environment, assumptions, limitations, parameters, and implementation details**.

E. Student Work

1. Problem Understanding and Formulation

1.1 Problem Description

Large-scale logistics companies handle thousands of packages every day. Each package has different characteristics such as weight, value, priority, and delivery deadline. Vehicles have limited carrying capacity. Therefore, selecting the most suitable packages for transportation is an optimization problem. The objective of this project is to select packages in such a way that the **total value of the selected packages is maximized without exceeding the vehicle capacity**.

Three approaches are implemented and compared:

1. Dynamic Programming baseline
2. Greedy baseline
3. Proposed Adaptive Priority-Density Logistics Optimizer (APDLO)

The assignment requires testing the algorithms under different package sizes and operating conditions.

1.2 Problem Formulation

Let:

- n = number of packages
- w_i = weight of package i
- v_i = value of package i
- p_i = priority of package i
- d_i = deadline of package i
- C = vehicle capacity

Define:

$$x_i = \begin{cases} 1 & \text{if package } i \text{ is selected} \\ 0 & \text{otherwise} \end{cases}$$

The objective is:

$$\text{Maximize } Z = \sum_{i=1}^n v_i x_i$$

Subject to:

$$\sum_{i=1}^n w_i x_i \leq C$$

and

$$x_i \in \{0, 1\}$$

The main goal is to maximize transportation value while satisfying vehicle capacity.

2. Application of Course Knowledge

2.1 Dynamic Programming

Dynamic Programming is used as the baseline optimization method.

The DP state is:

$$DP[i][c]$$

where i represents the number of packages considered and c represents the available vehicle capacity.

The recurrence is:

$$DP[i][c] = \max(D, P[i-1][c]v_i + DP[i-1][c - w_i])$$

when the package can fit.

If the package cannot fit:

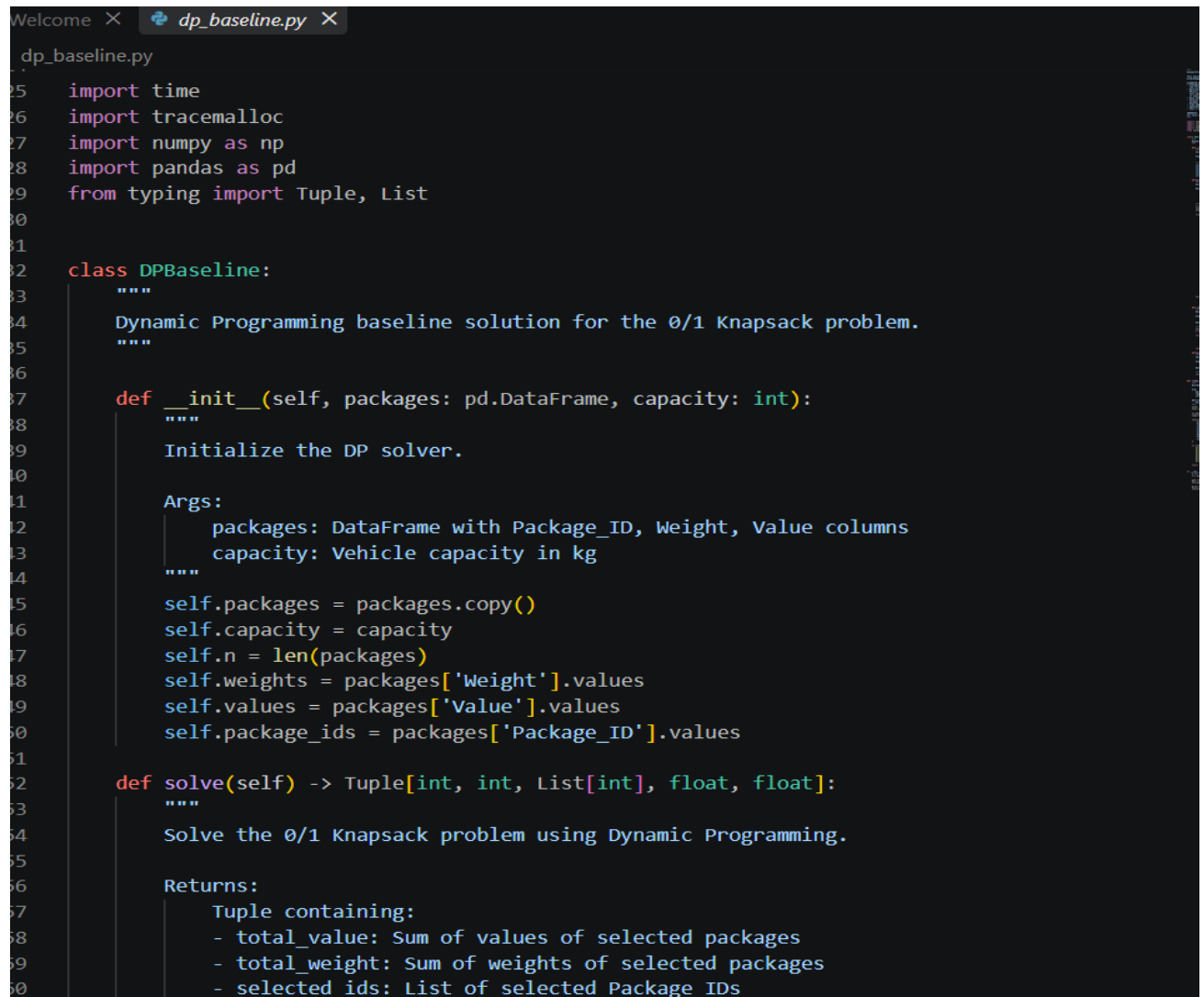
$$DP[i][c] = DP[i-1][c]$$

Time Complexity

$$O(nC)$$

Space Complexity

$$O(nC)$$



```
dp_baseline.py
25 import time
26 import tracemalloc
27 import numpy as np
28 import pandas as pd
29 from typing import Tuple, List
30
31
32 class DPBaseline:
33     """
34     Dynamic Programming baseline solution for the 0/1 Knapsack problem.
35     """
36
37     def __init__(self, packages: pd.DataFrame, capacity: int):
38         """
39         Initialize the DP solver.
40
41         Args:
42             packages: DataFrame with Package_ID, Weight, Value columns
43             capacity: Vehicle capacity in kg
44         """
45         self.packages = packages.copy()
46         self.capacity = capacity
47         self.n = len(packages)
48         self.weights = packages['Weight'].values
49         self.values = packages['Value'].values
50         self.package_ids = packages['Package_ID'].values
51
52     def solve(self) -> Tuple[int, int, List[int], float, float]:
53         """
54         Solve the 0/1 Knapsack problem using Dynamic Programming.
55
56         Returns:
57             Tuple containing:
58             - total_value: Sum of values of selected packages
59             - total_weight: Sum of weights of selected packages
60             - selected_ids: List of selected Package_IDs
61         """
```

2.2 Greedy Algorithm

The Greedy algorithm selects packages according to their value density.

The density is calculated as:

$$Density_i = \frac{Value_i}{Weight_i}$$

Packages are sorted in decreasing order of density. The algorithm selects a package if it does not exceed the remaining vehicle capacity.

Time Complexity

$$O(n \log n)$$

because packages are sorted according to density.

Space Complexity

$$O(n)$$

```
import time
import tracemalloc
import numpy as np
import pandas as pd
from typing import List, Tuple

class GreedyBaseline:
    """
    Greedy baseline solution using value-to-weight density heuristic.
    """

    def __init__(self, packages: pd.DataFrame, capacity: int):
        """
        Initialize the Greedy solver.

        Args:
            packages: DataFrame with Package_ID, Weight, Value columns
            capacity: Vehicle capacity in kg
        """
        self.packages = packages.copy()
        self.capacity = capacity
        self.n = len(packages)

    def solve(self) -> Tuple[int, int, List[int], float, float]:
        """
        Solve using greedy density-based selection.

        Returns:
            Tuple containing:
            - total_value: Sum of values of selected packages
            - total_weight: Sum of weights of selected packages
            - selected_ids: List of selected Package_IDs
            - execution_time: Time taken to solve in seconds
            - peak_memory: Peak memory used in MB
        """
```

2.3 Proposed Solution – APDLO

The proposed algorithm is called:

Adaptive Priority-Density Logistics Optimizer (APDLO).

The algorithm considers multiple factors instead of only value density.

The factors are:

- Package value
- Package priority
- Deadline urgency

- Value density
- Remaining vehicle capacity

The basic score is calculated as:

$$Score_i = 0.40V_i + 0.25P_i + 0.20U_i + 0.15D_i$$

where:

- V_i = normalized value
- P_i = normalized priority
- U_i = deadline urgency
- D_i = value density

Deadline urgency is calculated as:

$$U_i = \frac{1}{Deadline_i}$$

Packages with higher scores are considered first.

The proposed solution also considers different operating conditions:

1. Normal Demand
2. High-Priority Demand
3. Capacity-Constrained Demand

After the initial selection, a local improvement step is used to check whether replacing a selected package with another package can improve the total value while maintaining capacity feasibility.

```
import time
import tracemalloc
import numpy as np
import pandas as pd
from typing import List, Tuple, Literal

class APDLO:
    """
    Adaptive Priority-Density Logistics Optimizer algorithm.
    """

    # Operating condition weight profiles
    WEIGHT_PROFILES = {
        'NORMAL': {
            'value': 0.40,
            'priority': 0.25,
            'urgency': 0.20,
            'density': 0.15
        },
        'HIGH_PRIORITY': {
            'value': 0.30,
            'priority': 0.35,
            'urgency': 0.25,
            'density': 0.10
        },
        'CAPACITY_CONSTRAINED': {
            'value': 0.25,
            'priority': 0.15,
            'urgency': 0.15,
            'density': 0.45
        }
    }

    def __init__(self,
                 packages: pd.DataFrame,
```

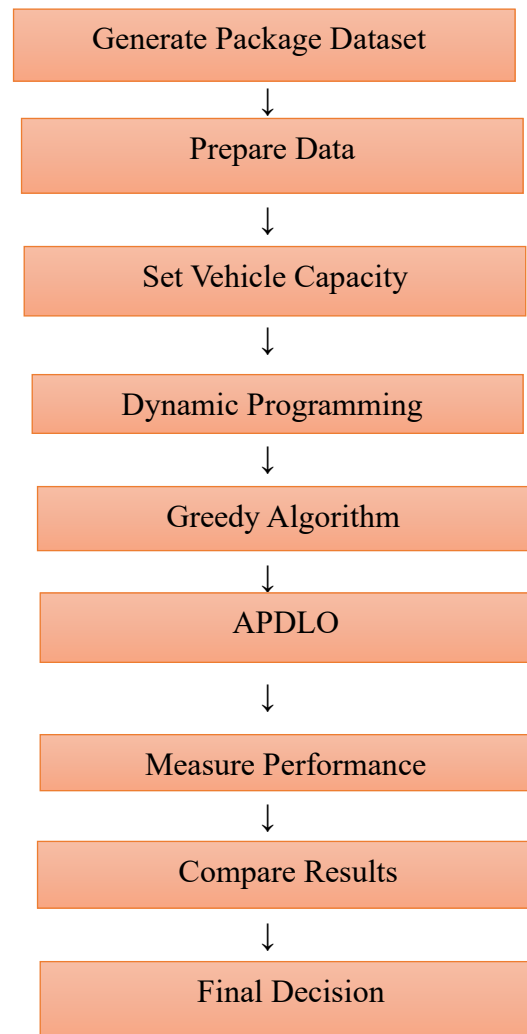
3. Solution Design and Methodology

The overall methodology is:

Input Dataset → Data Preparation → Problem Formulation → DP Baseline → Greedy Baseline → Limitation Analysis → APDLO Development → Implementation → Experimental Evaluation → Optimization → Performance Comparison → Final Decision

This follows the methodology structure specified in the assignment.

3.1 Proposed System Flow



3.2 Dynamic Programming Pseudocode

START

Read package weights and values

Read vehicle capacity

Initialize DP table

FOR each package:

 FOR each capacity:

 IF package weight $\leq$ capacity:

 Choose maximum of:

Exclude package
 Include package
ELSE:
 Exclude package

Return maximum value

END

3.3 Greedy Pseudocode

START

Read package data

Calculate value/weight density

Sort packages by decreasing density

FOR each package:

 IF package fits in remaining capacity:

 Select package

 Update total weight and value

Return selected packages

END

3.4 APDLO Pseudocode

START

Read package data

Calculate value density

Calculate deadline urgency

Normalize package attributes

Calculate APDLO score

Adjust score according to demand condition

Sort packages according to score

Select packages while capacity is available

Perform local improvement

Check feasibility

Return selected packages and total value

END

4. Implementation and Modern Tools

The project was implemented using Python in Visual Studio Code.

Software and Libraries

Tool	Purpose
Python	Algorithm implementation
VS Code	Development environment
NumPy	Dataset generation
Pandas	Data processing
Matplotlib	Graph generation
Time	Execution-time measurement
Tracemalloc	Memory measurement

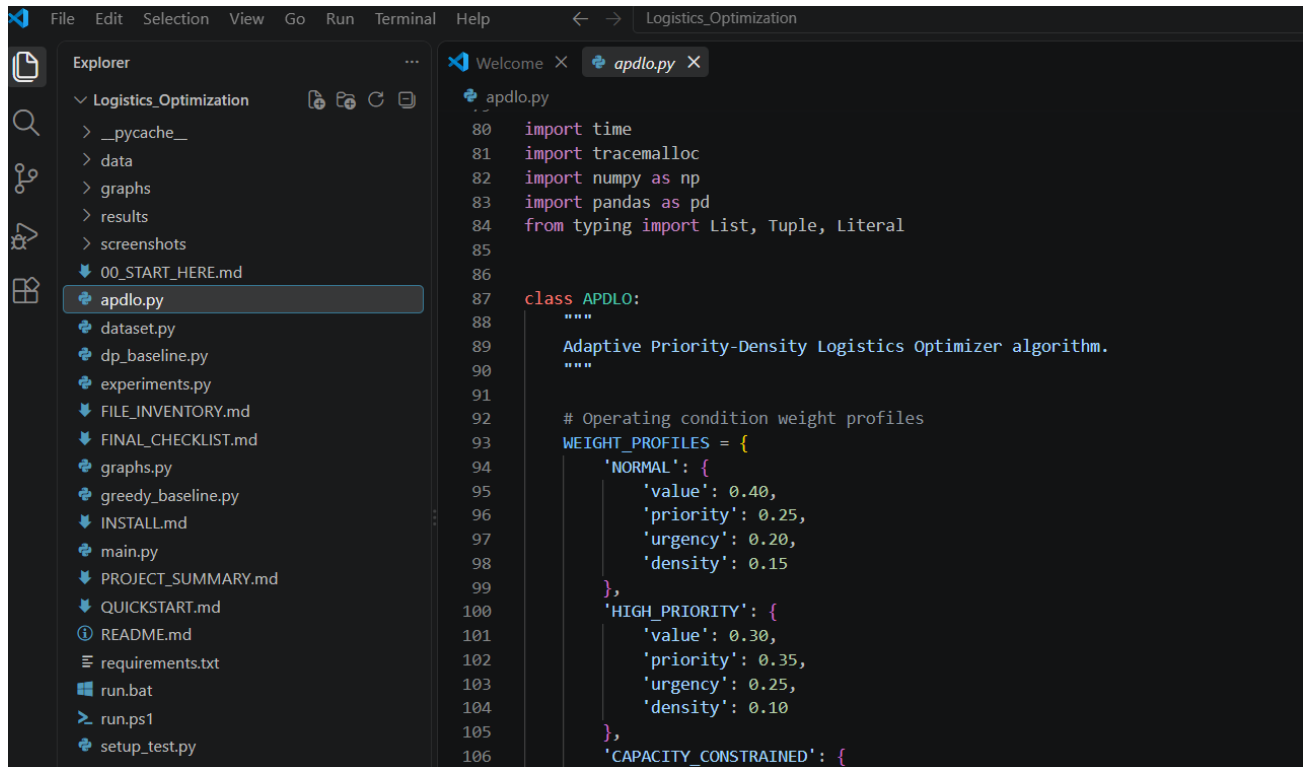
The assignment permits Python/C/C++/Java and expects evidence of implementation, setup, outputs, tables and graphs.

4.1 Project Structure

Your project contains:

Logistics_Optimization/

```
|
|— data/
|— graphs/
|— results/
|— screenshots/
|
|— dataset.py
|— dp_baseline.py
|— greedy_baseline.py
|— apdlo.py
|— experiments.py
|— graphs.py
|— main.py
|— requirements.txt
└— README.md
```



```
File Edit Selection View Go Run Terminal Help
Logistics_Optimization

Explorer
Logistics_Optimization
  __pycache__
  data
  graphs
  results
  screenshots
  00_START_HERE.md
  apdlo.py
  dataset.py
  dp_baseline.py
  experiments.py
  FILE_INVENTORY.md
  FINAL_CHECKLIST.md
  graphs.py
  greedy_baseline.py
  INSTALL.md
  main.py
  PROJECT_SUMMARY.md
  QUICKSTART.md
  README.md
  requirements.txt
  run.bat
  run.ps1
  setup_test.py

apdlo.py
80 import time
81 import tracemalloc
82 import numpy as np
83 import pandas as pd
84 from typing import List, Tuple, Literal
85
86
87 class APDLO:
88     """
89     Adaptive Priority-Density Logistics Optimizer algorithm.
90     """
91
92     # Operating condition weight profiles
93     WEIGHT_PROFILES = {
94         'NORMAL': {
95             'value': 0.40,
96             'priority': 0.25,
97             'urgency': 0.20,
98             'density': 0.15
99         },
100         'HIGH_PRIORITY': {
101             'value': 0.30,
102             'priority': 0.35,
103             'urgency': 0.25,
104             'density': 0.10
105         },
106         'CAPACITY_CONSTRAINED': {
```

5. Experimental Setup

The algorithms are tested using different input sizes and vehicle capacities.

Input Sizes

- 100 packages
- 500 packages
- 1,000 packages
- 5,000 packages
- 10,000 packages

Vehicle Capacities

- 500 kg
- 1,000 kg
- 2,500 kg
- 5,000 kg

Operating Conditions

1. Normal Demand
2. High-Priority Demand
3. Capacity-Constrained Demand

These experiment categories and metrics are required by the assignment.

6. Results and Validation

The three algorithms are evaluated using the same input data and environment.

The following metrics are measured:

- Total solution value
- Total weight
- Number of selected packages
- Capacity utilization
- Execution time
- Memory usage
- Optimality gap
- Input size
- Feasibility

The assignment specifically requires quantitative comparison using these types of measures.

```
=====
INPUT SIZE: 100 packages
VEHICLE CAPACITY: 50 kg
OPERATING CONDITION: NORMAL
=====

Dynamic Programming:
  Total Value: 2397
  Selected Packages: 7
  Total Weight: 50 kg
  Capacity Utilization: 100.00%
  Execution Time: 0.050034 seconds
  Memory Usage: 0.04 MB
  Feasible: ✓

Greedy (Density-based):
  Total Value: 2397.0
  Selected Packages: 7
  Total Weight: 50.0 kg
  Capacity Utilization: 100.00%
  Execution Time: 0.015391 seconds
  Memory Usage: 0.03 MB
  Feasible: ✓

APDLO (Adaptive Priority-Density):
  Total Value: 1880
  Selected Packages: 4
  Total Weight: 49 kg
  Capacity Utilization: 98.00%
  Execution Time: 0.035419 seconds
  Memory Usage: 0.07 MB
  Feasible: ✓

Optimality Gap (APDLO vs DP): 21.57%
=====
```

6.1 Sample Results Table

IMPORTANT: Replace the numbers below with the **actual numbers produced by your program**.

Algorithm	Total Value	Total Weight	Selected Packages	Capacity Utilization	Execution Time	Memory
Dynamic Programming	Actual	Actual	Actual	Actual	Actual	Actual
Greedy	Actual	Actual	Actual	Actual	Actual	Actual
APDLO	Actual	Actual	Actual	Actual	Actual	Actual

6.2 Comparison Graphs

Generate graphs for:

1. Execution Time
2. Memory Usage
3. Total Solution Value
4. Capacity Utilization
5. Optimality Gap

The assignment asks for tables/graphs to support the comparison.

7. Analysis and Engineering Decision

Dynamic Programming provides a strong baseline for obtaining a high-quality solution, but its computational cost increases with the capacity and number of packages.

The Greedy algorithm is computationally efficient because it sorts packages according to value density. However, it may not always produce the best overall solution because it makes decisions based on a local criterion.

APDLO considers multiple package characteristics such as value, priority, urgency and density. Therefore, it is designed to provide a better balance between solution quality and computational efficiency.

The experimental results are used to determine which algorithm provides the most suitable trade-off between:

- Solution quality
- Execution time
- Memory usage
- Capacity utilization
- Scalability

The assignment requires the final engineering decision to be supported by quantitative experimental evidence.

8. Broader Considerations and SDG 9

The proposed logistics optimization system can help improve transportation resource utilization.

Efficient package selection can:

- Improve vehicle capacity utilization.
- Reduce computational wastage.
- Support large-scale logistics operations.
- Improve resource allocation.
- Help prioritize urgent packages.
- Support scalable transportation planning.

The project is related to **Sustainable Development Goal 9 (Industry, Innovation and Infrastructure)** because it applies algorithmic optimization and computational methods to improve logistics infrastructure and resource utilization.

9. Conclusion

This project developed an intelligent algorithmic solution for large-scale logistics resource optimization.

Dynamic Programming, Greedy and APDLO algorithms were implemented and evaluated using common datasets and experimental conditions.

Dynamic Programming provides a strong solution-quality baseline. Greedy provides fast decision-making using value density. APDLO combines multiple package characteristics to make more informed selection decisions.

The final algorithm should be selected based on the **actual experimental results** considering solution value, execution time, memory usage, capacity utilization and scalability.

10. Student Reflection

10.1 What did you learn from this project?

I learned how optimization problems can be formulated mathematically and solved using Dynamic Programming and Greedy algorithms. I also learned how to develop and evaluate an improved algorithm.

10.2 What was the most challenging part?

The most challenging part was designing the APDLO scoring strategy and comparing the algorithms under different operating conditions.

10.3 How did you improve your problem-solving skills?

I learned how to divide a large optimization problem into smaller components, analyze algorithmic complexity and use experimental results to make engineering decisions.

10.4 How can this solution be improved?

The solution can be improved by using larger real-world logistics datasets, multiple vehicle constraints, route information, real-time traffic information and more advanced optimization techniques.

11. References

Include:

1. Your dataset source.
2. Dynamic Programming/Knapsack algorithm reference.
3. Greedy algorithm reference.
4. Python documentation.
5. NumPy documentation.
6. Pandas documentation.
7. Matplotlib documentation.
8. Any research papers used.
9. AI-assisted tools used during development.

The assignment specifically asks references to include datasets, algorithms/research, software/tools and AI-assisted tools where applicable.

F. Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	10	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.

Algorithmic Solution Design	15	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization of Algorithmic Solution	20	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.
Implementation & Modern Tool Usage	15	Developed solution is correctly and efficiently implemented using appropriate programming/computational tools and handles relevant input conditions reliably.	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable or non-functional.
Efficiency, Testing & Validation	15	Performs appropriate complexity evaluation and systematic testing using suitable data/input sizes; validates correctness, efficiency and scalability with clear evidence.	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.
Performance Improvement, Comparison & Final Decision	15	Demonstrates the effectiveness of the developed solution through appropriate comparison; critically evaluates performance,	Good performance comparison and justification	Basic comparison is provided, but improvement or final	Improvement is not demonstrated or the final algorithmic
		limitations and trade-offs and justifies the final algorithmic decision with evidence.	with minor limitations.	justification lacks depth.	decision is unsupported.

Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development, SDG relevance, challenges, or learning outcomes.
Total	100				

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Understanding & Algorithmic Formulation	CO4	PO1, PO2	L4	10
Algorithmic Solution Design	CO4	PO2, PO3	L4, L6	15
Development / Adaptation / Optimization of Algorithmic Solution	CO4	PO2, PO3	L5, L6	20
Implementation & Modern Tool Usage	CO4	PO3, PO5	L4, L6	15
Efficiency, Testing & Validation	CO4	PO2, PO4	L4, L5	15
Performance Improvement, Comparison & Final Decision	CO4	PO2, PO4	L4, L5	15
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	CO4	PO7, PO10, PO12	L4, L5	10

H. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action: